Rapport de Sprint

Developpement de Modules Drupal

Duree du Sprint : 5 Jours

Domaines : Module Development · Routes & Services · Hooks · Plugins & Forms · Data Types

Nom : Sadiki Abdelkarim

Date de redaction : 9 mars 2026

Type de rapport : Rapport technique de sprint

Plateforme : Drupal CMS (v10+)

Outils utilises : Composer · Drush · Git · PHPCS · PHPStan

Table des matières

Introduction **2**

1 Jour 1 — Bases du Developpement de Modules **3**
1.1 Contexte et Objectifs . . . 3
1.2 Apprentissages Theoriques . . . 3
1.3 Prerequis et Documentation . . . 3
1.4 Exercices Pratiques . . . 3
1.4.1 Creation des premiers modules . . . 3
1.4.2 Tests de comportement du module . . . 3
1.4.3 Creation d'un bloc personnalise . . . 3
1.5 Livrables . . . 4

2 Jour 2 — Routes, Controleurs, Services & Injection de Dependances **5**
2.1 Contexte et Objectifs . . . 5
2.2 Concepts Etudies . . . 5
2.3 Questions de Pratique et Reponses . . . 5
2.3.1 Navigation et Menus . . . 5
2.3.2 Controleurs et Services . . . 5
2.3.3 Traduction . . . 5
2.3.4 Exercice Drush . . . 5

3 Jour 3 — Hooks **6**
3.1 Contexte et Objectifs . . . 6
3.2 Concepts Cles . . . 6
3.3 Exercices Realises . . . 6
3.3.1 Exploration des hooks du module Metatag . . . 6
3.3.2 Alteration de formulaire avec hook_form_alter . . . 6
3.3.3 Variable is_front dans les templates Twig . . . 6
3.3.4 Ajout d'une balise meta via hook_page_attachments_alter . . . 6
3.3.5 Autres hooks pratiques . . . 7

4 Jour 4 — Plugins & Formulaires **8**
4.1 Contexte et Objectifs . . . 8
4.2 Concepts Etudies . . . 8
4.3 Questions et Reponses — Form API . . . 8
4.3.1 Validation des donnees . . . 8
4.3.2 Affichage d'un formulaire dans un Bloc . . . 8
4.3.3 Redirection et message apres soumission . . . 8
4.3.4 Controle d'acces et groupement de champs . . . 8

5 Jour 5 — Types de Donnees **9**
5.1 Contexte et Objectifs . . . 9
5.2 Concepts Etudies . . . 9
5.3 Questions et Reponses . . . 9
5.4 Exercice Pratique Final . . . 9

Introduction

Ce document constitue le rapport de sprint couvrant **5 journees** de formation intensive au developpement Drupal. Chaque journee cible un domaine technique precis, de la creation de modules jusqu'a la gestion avancee des types de donnees, en passant par les routes, les hooks, et les formulaires.

Objectifs du Sprint

- Acquerir les bases solides du developpement de modules Drupal
- Maitriser le systeme de routes, services et injection de dependances
- Comprendre et utiliser le systeme de hooks de Drupal
- Manipuler les plugins, les formulaires et l'API Form
- Gerer les types de donnees, entites et requetes d'entites

1. Jour 1 — Bases du Developpement de Modules

1.1. Contexte et Objectifs

La premiere journee pose les fondations du developpement Drupal. L'objectif est de comprendre la structure d'un module Drupal et d'en creer un premier fonctionnel.

1.2. Apprentissages Theoriques

Les six domaines cles abordes lors de la phase de visionnage :

1. **Gestion des paquets** avec Composer
2. **Structure d'un module** et bases du developpement
3. **Systeme de hooks et evenements**
4. **Formulaires et rendu**
5. **Bonnes pratiques** de developpement
6. **Exemples pratiques** de modules

1.3. Prerequis et Documentation

Lecture des ressources officielles Drupal couvrant :

— Prerequis et contexte de creation de modules
— Nommage et placement du module dans le projet
— Fichier `info.yml` pour declarer un module aupres de Drupal
— Activation du logging verbose pour le debogage

1.4. Exercices Pratiques

Creation des premiers modules

— Desactivation du cache Drupal pour l'environnement de developpement
— Creation d'un premier module *Hello World* selon le tutoriel de base
— Respect des conventions de nommage de modules
— Utilisation de **Git** pour le suivi des modifications
— Application de **PHPCS** et **PHPStan** pour la qualite du code
— Redaction de la documentation PHP selon les standards Drupal

Tests de comportement du module

Creation d'un bloc personnalise

Creation d'un bloc custom *Hello World* en suivant le tutoriel Drupalize.me sur l'anatomie d'un module.

Test	Observation attendue
Desinstallation / Reinstallation	Verification de la persistance des donnees
Modification de `core_version_requirement`	Message d'incompatibilite de version
Ajout de `workflows` comme dependance	Gestion des dependances manquantes
Ajout d'un `composer.json`	Integration du module dans l'ecosysteme Composer
Suppression / deplacement du module	Analyse des logs avec `drush ws`

Table 1 – Scenarios de test du module

1.5. Livrables

- Deux modules fonctionnels crees et versionnes sous Git
- Bloc personnalise operationnel
- Modules publies dans le depot Git partage

2. Jour 2 — Routes, Controleurs, Services & Injection de Dependances

2.1. Contexte et Objectifs

Cette journee approfondit l'architecture Drupal a travers le systeme de routage, les controleurs, et le conteneur de services (injection de dependances).

2.2. Concepts Etudies

- **Routes** : definition, parametres, poids et menus hierarchiques
- **Services & Conteneur** : principes de l'injection de dependances
- **Sortie thematisable** : retour de templates Twig depuis un controleur
- **Passage de donnees PHP vers JavaScript** dans Drupal

2.3. Questions de Pratique et Reponses

Navigation et Menus

- Controle de l'ordre des menus via la propriete **weight** dans le fichier `.links.menu.yml`
- Configuration des menus enfants via la propriete **parent**

Controleurs et Services

- Recuperation d'un query string via `$request->query->get('param')`
- **Guzzle** : client HTTP integre pour les requetes externes
- **Logger** : service de journalisation, logs visibles dans `/admin/reports/dblog`
- Fichier des services du core : `core.services.yml`
- Injection de services via la methode `create()` et le constructeur
- Retour d'un template Twig via un render array avec `#theme`
- Ajout de JS externe via les **libraries** dans `.libraries.yml`
- Reponse JSON avec `JsonResponse` (Content-Type automatique)

Traduction

- Recherche de `Hello, @name!` dans l'interface de traduction Drupal
- Traduction des chaines JavaScript avec `Drupal.t()`

Exercice Drush

Listing 1 – Utilisation du service path.alias_manager via Drush

```
$ drush php
>>> $alias_manager = \Drupal::service('path_alias.manager');
>>> $alias = $alias_manager->getAliasByPath('/node/1');
>>> print($alias);
```

3. Jour 3 — Hooks

3.1. Contexte et Objectifs

Le systeme de hooks est au coeur de l'extensibilite de Drupal. Cette journee est consacree a la comprehension et a l'utilisation pratique des hooks.

3.2. Concepts Cles

- Definition et role des hooks dans l'architecture evenementielle de Drupal
- Declaration d'un hook dans un fichier `.module`
- Hooks de pretraitement (*preprocess hooks*) pour la couche de theme

3.3. Exercices Realises

Exploration des hooks du module Metatag

Identification des hooks fournis par le module **metatag** dans ses fichiers `.api.php`.

Alteration de formulaire avec hook_form_alter

- Hook utilise : `hook_form_alter(&$form, FormStateInterface $form_state, $form_id)`
- Arguments recus : tableau `$form`, etat du formulaire, identifiant du formulaire
- Ajout d'un champ texte personnalise dans un formulaire existant

Variable is_front dans les templates Twig

Utilisation du service `path.matcher` et d'un hook `preprocess_page` pour exposer la variable `is_front` aux templates, notamment `page.html.twig`.

Ajout d'une balise meta via hook_page_attachments_alter

Listing 2 – Ajout d'une balise viewport

```
function mymodule_page_attachments_alter(array &$attachments) {
  $attachments['#attached']['html_head'][] = [
    [
      '#tag' => 'meta',
      '#attributes' => [
        'name' => 'viewport',
        'content' => 'width=device-width, initial-scale=1',
      ],
    ],
    'viewport',
  ];
}
```

Autres hooks pratiques

- `hook_preprocess_menu` : ajout de la classe `.my-custom-class` a tous les elements de menu
- `hook_preprocess_block` : remplacement du logo du bloc `system_branding_block` par le logo Drupal Wordmark

4. Jour 4 — Plugins & Formulaires

4.1. Contexte et Objectifs

Cette journee aborde deux piliers essentiels : le systeme de plugins de Drupal et l'API des formulaires (*Form API*).

4.2. Concepts Etudies

- Architecture des **plugins** Drupal (annotations, decouverte automatique)
- **Form API** : structure d'un formulaire, types de champs, validation, soumission
- Modules de reference : `form_api_example`, `file_example`

4.3. Questions et Reponses — Form API

Validation des donnees

La validation s'effectue dans la methode `validateForm()` du formulaire. Elle peut aussi etre etendue via le hook `hook_form_alter` ou en ajoutant des `#element_validate`.

Affichage d'un formulaire dans un Bloc

Listing 3 – Rendu d'un formulaire dans la methode build() d'un bloc

```
public function build() {
  $form = \Drupal::formBuilder()->getForm(MyCustomForm::class);
  return $form;
}
```

Redirection et message apres soumission

- Redirection via `$form_state->setRedirect('route.name')` dans `submitForm()`
- Message de confirmation via `\Drupal::messenger()->addStatus($this->t('...'))`

Controle d'acces et groupement de champs

- Propriete `#access` : `FALSE` pour masquer un champ aux utilisateurs anonymes
- Groupement de champs avec `#type => 'fieldset'` ou `'details'`, similaire au module `field_group`

5. Jour 5 — Types de Donnees

5.1. Contexte et Objectifs

La derniere journee couvre la gestion des donnees dans Drupal : configuration, entites, requetes et traductions.

5.2. Concepts Etudies

- **Configuration API** : fichiers YAML, schemas, import/export
- **Entites** : chargement, lecture et mise a jour de champs
- **Entity Queries** : requetes programmatiques sur les entites
- **Contraintes** : validation de donnees d'entite
- **View Builders** : modes d'affichage (teaser, full, etc.)
- **Traductions** : recuperation d'une entite dans une langue donnee

5.3. Questions et Reponses

Question	Reponse
Localisation du fichier de config `eu_cookie_compliance`	`config/install/` du module
Role du fichier `schema.yml`	Declare la structure et types des configs pour validation
Chargement d'un noeud	`Node::load($nid)` ou via `entityTypeManager`
Mise a jour d'un champ	`$node->set('field', $val);` `$node->save();`
Definition d'une Contrainte	Classe de validation attachee a un type de champ
Role de `accessCheck`	Applique ou ignore les controles d'acces dans les requetes
Traduction d'un noeud	`$node->getTranslation('fr')`

Table 2 – Synthese des reponses — Types de donnees

5.4. Exercice Pratique Final

1. **Formulaire avec champ entity reference** vers des noeuds
2. **Bloc d'affichage** du titre du noeud selectionne
3. **Requete d'entite** pour lister les noeuds du meme type de contenu (en excluant le noeud courant)

4. **Hook `hook_theme`** + template Twig pour afficher titre et liste de titres

Listing 4 – Exemple de requete d'entite

```
$query = \Drupal::entityQuery('node')
  ->condition('type', $bundle)
  ->condition('nid', $current_nid, '<>')
  ->condition('status', 1)
  ->accessCheck(TRUE);
$nids = $query->execute();
$nodes = Node::loadMultiple($nids);
```